Sampling Better Neural Cellular Automata for Pretraining

Sagnik Bhattacharjee
sbcharjee.acad@gmail.com @datavorous_

Draft

Abstract

Pre-pretraining language models on neural cellular automata (NCA) trajectories can speed up later language training. We ask whether choosing which NCA worlds to train on matters beyond filtering them by gzip complexity. Using a 1M parameter Transformer on TinyStories, we reproduce the NCA transfer benefit and find that most of it remains after shuffling the NCA tokens. We then test several simple properties of NCA trajectories and find that worlds with higher horizontal recurrence transfer better, while selecting for lower recurrence hurts. A stronger recurrence filter reduces the downstream tokens needed to reach the same validation loss by about 17% compared with ordinary NCA.

1 Introduction

Lee et al. [4] show that language models can benefit from pre-pretraining on trajectories generated by neural cellular automata (NCA) before seeing natural language. They report faster downstream convergence and lower language modeling loss, and find that the best NCA complexity depends on the target domain. Their main control for choosing trajectories is gzip compressibility [1].

Compressibility is useful, but coarse. Two trajectories can have similar gzip complexity while exposing a Transformer to very different local structure. We therefore ask a narrower question: among NCA trajectories with similar complexity, are some better pretraining data than others?

We study this at small scale with a 1M parameter Transformer. We keep the NCA generator, token budget, gzip range, and model fixed, and change only which trajectories are selected for pretraining. We first reproduce the NCA transfer effect, then test whether simple properties of the generated trajectories predict better downstream learning.

2 Setup

NCA data We follow the NCA generation and tokenization setup of Lee et al. [4]. Each world is a 12×12 grid with 10 cell states. We first let each NCA evolve for 10 steps. We record 27 frames and split each into nonoverlapping 2×2 patches. Each frame contains 36 patches and two frame markers, giving 1,026 tokens per world. We group trajectories into four gzip complexity bands: $[.2, .3)$, $[.3, .4)$, $[.4, .5)$, and $[.5, \infty)$. Each NCA pretraining run uses 19.7M scored tokens.

Model and transfer We use a small causal Transformer [8] with 4 layers, 2 attention heads, width 20, and context length 256. It has 1.08M parameters with the language vocabulary. After NCA pretraining, we discard the NCA embeddings and output head, transfer the Transformer body, and initialize a new language embedding and tied output head. We then train on 10M TinyStories tokens [2]. All paired conditions use the same data order, optimization schedule, and model initialization. We evaluate five model seeds unless stated otherwise.

Evaluation Our primary measurement is validation loss at fixed TinyStories token counts. We also report downstream sample efficiency as the number of language tokens needed to reach the loss obtained by the corresponding scratch model after 10M tokens. Crossings between evaluation checkpoints are linearly interpolated. This metric counts language tokens only and does not include the cost of NCA pretraining.

3 Results

3.1 Shuffling Preserves Transfer

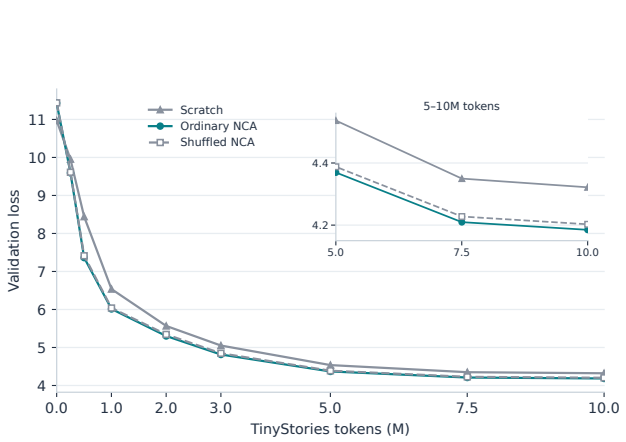
All four NCA bands outperform Scratch (language training without NCA pretraining) at 10M TinyStories tokens (Fig. 1, Table 1). The 40-50% gzip band performs best on average, with validation loss 4.185 ± 0.024 compared with 4.322 ± 0.020 for Scratch. This corresponds to a downstream-token speedup of $1.77 \pm 0.21\times$.

We use Ordinary NCA (unfiltered worlds from the 40–50% gzip band) as the reference condition for later experiments.

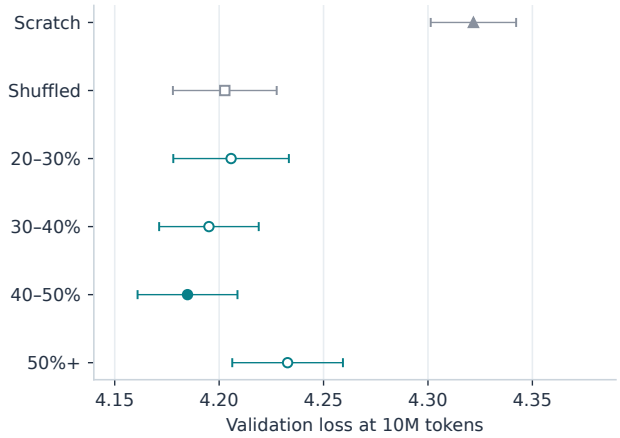
The differences between the middle gzip bands are small. The 40-50% band is only 0.010 lower in loss than the 30-40% band, and it has the best final loss in 2/5 seeds. We therefore do not find a clear optimal gzip band.

The NCA advantage is largest early in language training. At 0.5M tokens, the gap to scratch is about 1.08 loss. By 10M tokens, it has narrowed to roughly 0.09-0.14.

We then shuffle the patch tokens inside each NCA trajectory while keeping the same tokens. The shuffled control reaches 4.203 ± 0.025 validation loss. The best structured band is only 0.018 lower. Most of the NCA transfer benefit therefore survives shuffling.



TinyStories validation loss for Scratch, shuffled NCA, and ordinary NCA, averaged over five seeds. The inset shows the final 5M tokens.



Final validation loss for Scratch, shuffled NCA, and four gzip bands. Points show seed means; whiskers show one standard deviation.

Figure 1: NCA pretraining improves over Scratch, while differences between gzip bands remain small.

Table 1: Validation loss after 10M TinyStories tokens. Speedup measures how quickly each condition reaches the final loss of its paired scratch model.

Condition	Val. loss	PPL	Gain vs. scratch	Speedup
Scratch	4.322 ± 0.020	75.33	–	$1.00\times$
Shuffled NCA	4.203 ± 0.025	66.88	0.119 ± 0.043	$1.68 \pm 0.21\times$
20-30% gzip	4.206 ± 0.028	67.09	0.116 ± 0.041	$1.64 \pm 0.21\times$
30-40% gzip	4.195 ± 0.024	66.38	0.127 ± 0.009	$1.70 \pm 0.06\times$
40-50% gzip	4.185 ± 0.024	65.70	0.137 ± 0.039	$1.77 \pm 0.21\times$
50%+ gzip	4.233 ± 0.026	68.93	0.089 ± 0.039	$1.56 \pm 0.15\times$

3.2 Recurrence Helps

We next ask whether simple properties of an NCA trajectory can predict better transfer. We test six ways of selecting worlds within the 40-50% gzip band. Changes in cell activity, spatial agreement, and transition mutual information do not improve downstream learning.

One property does. Some worlds repeat horizontally adjacent 2×2 patches more often than expected from their overall patch frequencies.

We call this horizontal recurrence. It gives each world a recurrence score that can be computed before language model training. Each frame contains $6 \times 6 = 36$ nonoverlapping patches and $6 \times 5 = 30$ horizontal patch pairs, giving $N = 27 \times 36 = 972$ patches and 810 horizontal pairs per world. Let $p_{t,r,c}$ denote the patch ID at frame t, row r, and column c, and let n_v be the number of occurrences of patch ID v.

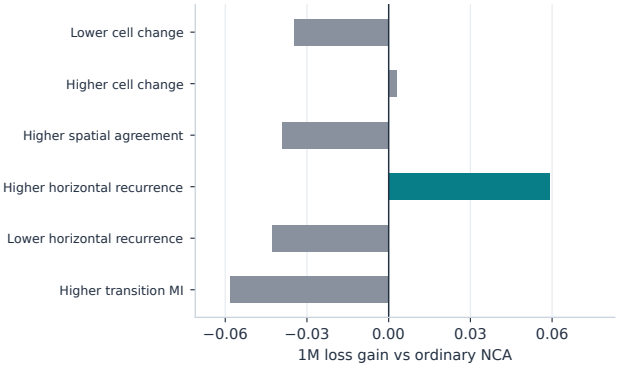
$$R_h = \frac{1}{810} \sum_{t=1}^{27} \sum_{r=1}^{6} \sum_{c=2}^{6} \mathbf{1}[p_{t,r,c} = p_{t,r,c-1}] - \frac{\sum_v n_v(n_v - 1)}{N(N-1)}.$$

The first term measures how often horizontally neighboring patches match. The second subtracts the match rate expected from the same patch counts under random ordering.

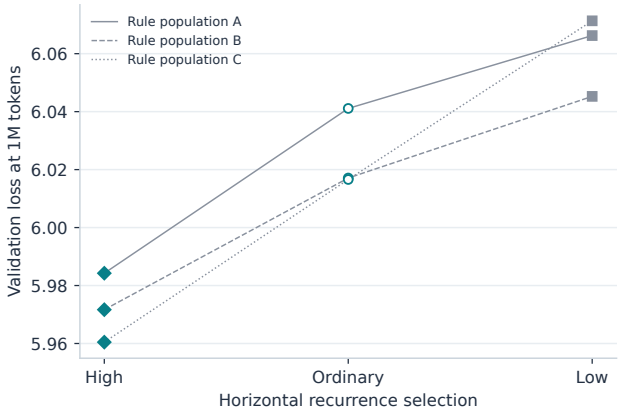
Selecting worlds with higher horizontal recurrence improves the loss after 1M TinyStories tokens by 0.059 ± 0.030 across five seeds. Selecting worlds with lower recurrence makes it worse by 0.043 ± 0.015.

We first found this pattern on one corpus, so we repeated the experiment on three new sets of NCA rules. Each set contains high recurrence, ordinary, and low recurrence worlds, evaluated with five model seeds. High recurrence beats ordinary NCA in 14/15 paired runs, and its average effect is positive in all three rule populations. Ordinary NCA beats low recurrence in 12/15 runs (Fig. 2).

Across the three new rule populations, high recurrence improves the 1M token loss by 0.053 ± 0.006 on average. Since these three populations are the independent replications, we treat this as consistent directional evidence rather than statistical significance.



Effects of trajectory selectors after 1M TinyStories tokens. Positive values mean lower loss than ordinary NCA.



Mean validation loss for high recurrence, ordinary NCA, and low recurrence across three independent rule populations. Lower is better.

Figure 2: Trajectory selection across independently generated NCA corpora.

What does recurrence capture? Horizontal patch recurrence is strongly related to other properties of the NCA dynamics, including lag-2 recurrence and transition mutual information. These features move together under our selector. We therefore treat recurrence as a useful way to select worlds, not as an isolated causal mechanism.

3.3 Stronger Selection

We next strengthen the recurrence filter. We choose its cutoff on the discovery data and freeze it before the confirmation experiment.

We test the stronger filter on three independently generated NCA corpora. For each corpus, Ordinary and Designed (top 10% by horizontal recurrence) use the same gzip distribution and NCA token budget. We train both conditions with five model initializations.

Across the three corpora, Designed outperforms Ordinary. At 10M TinyStories tokens, it lowers validation loss by 0.050 ± 0.014 on average across the three corpus means (Table 2).

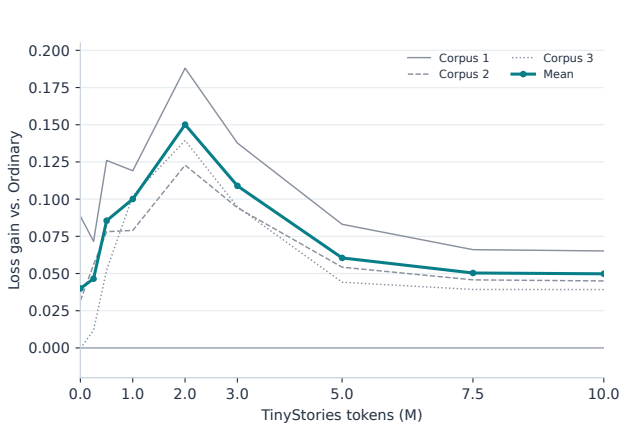
The same pattern appears in convergence speed. Ordinary reaches the paired Scratch endpoint with a mean speedup of $1.647 \pm 0.045\times$, while Designed reaches it at $1.983 \pm 0.041\times$. This corresponds to about 6.07M TinyStories tokens for

Table 2: Results after 10M TinyStories tokens.

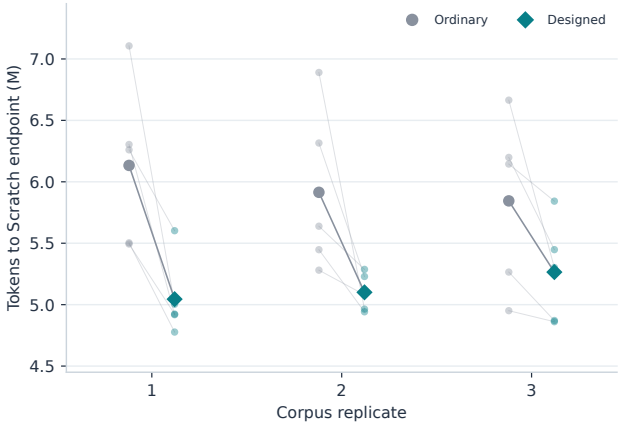
Condition	Val. loss	PPL	Gain vs. scratch	Gain vs. Ordinary	Speedup
Scratch	4.322 ± 0.020	75.33	–	–	$1.000\times$
Ordinary NCA	4.199 ± 0.007	66.64	0.123	–	$1.647 \pm 0.045\times$
Designed NCA	4.149 ± 0.007	63.39	0.173	0.050 ± 0.014	$1.983 \pm 0.041\times$

Ordinary and 5.04M for Designed, or about 17% fewer downstream tokens.

The advantage appears early in training (Fig. 3). The mean validation loss improvement peaks at about 0.15 near 2M TinyStories tokens, then gradually narrows. It remains positive through 10M tokens.



Designed gain over Ordinary during training. Thin lines show corpus means; the green line shows their overall mean.



Tokens to the paired Scratch endpoint. Small points show model runs; large points show corpus means. Fewer is better.

Figure 3: Effect of stronger recurrence selection.

4 Related Work

Language models can benefit from training on structured data that is not natural language [7]. Prior work has explored music [6], synthetic tasks [9], and formal languages [3] as sources of useful pretraining structure.

Lee et al. [4] extend this idea to trajectories generated by neural cellular automata [5]. They show that NCA pre-pretraining can improve later language training [10], and that the best NCA complexity depends on the downstream domain. We follow their released generation and tokenization setup.

5 Limitations

We study only a 1M parameter Transformer on TinyStories and use one NCA generator. Whether the same selection strategy helps larger models or other downstream datasets remains open.

The recurrence score is also not an isolated mechanism. Worlds with more horizontal recurrence differ in other ways, including lag-2 recurrence and transition mutual information. Our experiments therefore show that recurrence is useful for selecting NCA worlds, not that patch repetition itself causes better transfer.

Finally, our speedup metric counts only downstream language tokens and estimates crossings between evaluation checkpoints. We therefore treat it as a summary of learning speed rather than total training efficiency.

6 Conclusion

In our experiments, we found that the choice of NCA worlds matters. Selecting trajectories with more horizontal recurrence improves downstream language learning beyond gzip based filtering. The same direction appears across newly generated rule populations.

More broadly, our results suggest that NCA sampling itself can be hill-climbed. The space of generated worlds contains useful structure, and simple selection rules can move the training distribution toward worlds that transfer better.

References

[1] L. Peter Deutsch. GZIP file format specification version 4.3. RFC 1952, Internet Engineering Task Force, 1996.

[2] Ronen Eldan and Yuanzhi Li. TinyStories: How small can language models be and still speak coherent English? *arXiv preprint arXiv:2305.07759*, 2023.

[3] Michael Y. Hu, Jackson Petty, Chuan Shi, William Merrill, and Tal Linzen. Between circuits and chomsky: Pre-pretraining on formal languages imparts linguistic biases. *arXiv preprint arXiv:2502.19249*, 2025.

[4] Dan Lee, Seungwook Han, Akarsh Kumar, and Pulkit Agrawal. Training language models via neural cellular automata. *arXiv preprint arXiv:2603.10055*, 2026.

[5] Alexander Mordvintsev, Ettore Randazzo, Eyvind Niklasson, and Michael Levin. Growing neural cellular automata. *Distill*, 2020. doi: 10.23915/distill.00023.

[6] Isabel Papadimitriou and Dan Jurafsky. Learning music helps you read: Using transfer to study linguistic structure in language models. *arXiv preprint arXiv:2004.14601*, 2020.

[7] Zachary Shinnick, Liangze Jiang, Hemanth Saratchandran, Anton van den Hengel, and Damien Teney. Transformers pretrained on procedural data contain modular structures for algorithmic reasoning. *arXiv preprint arXiv:2505.22308*, 2025.

[8] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems 30*, 2017.

[9] Yuhuai Wu, Felix Li, and Percy Liang. Insights into pre-training via simpler synthetic tasks. In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, 2022.

[10] Atsuki Yamaguchi, Tatsuro Inaba, Joel Niklaus, Michal Štefánik, Aline Villavicencio, and Nikolaos Aletras. Synthetic pre-pretraining survives scale, but not as a grammatical prior. *arXiv preprint arXiv:2609.39827*, 2026.